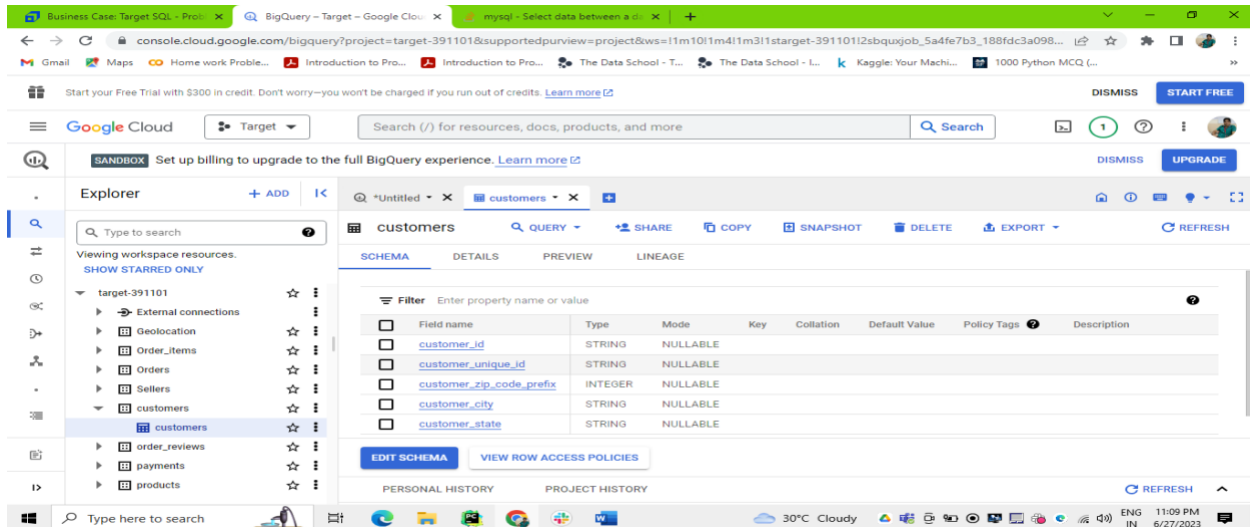


TARGET-BUSINESS ANALYSIS

Question-1: Import the dataset and do usual exploratory analysis steps like checking the structure & characteristics of the dataset:

A: Data type of all columns in the "customers" table.

Answer:



The screenshot shows the Google Cloud BigQuery console interface. On the left, the 'Explorer' pane displays a project hierarchy with 'customers' selected under 'target-391101'. The main area shows the 'customers' table schema with columns: customer_id (STRING, NULLABLE), customer_unique_id (STRING, NULLABLE), customer_zip_code_prefix (INTEGER, NULLABLE), customer_city (STRING, NULLABLE), and customer_state (STRING, NULLABLE). The 'Filter' section is empty, and the 'DETAILS' tab is active.

Field name	Type	Mode	Key	Collation	Default Value	Policy Tags	Description
customer_id	STRING	NULLABLE					
customer_unique_id	STRING	NULLABLE					
customer_zip_code_prefix	INTEGER	NULLABLE					
customer_city	STRING	NULLABLE					
customer_state	STRING	NULLABLE					

B: Get the time range between which the orders were placed.

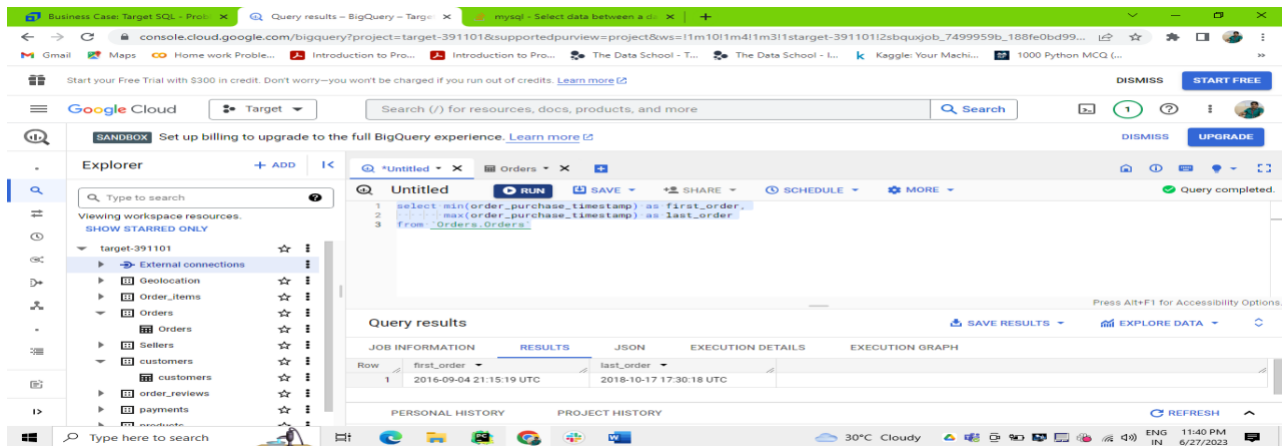
Answer:

The time range between which the orders were placed is as follows:

- The first order was placed on 4th September 2016
- The last order was placed on 17th October 2018

The query is as follows:

```
select min(order_purchase_timestamp) as first_order,  
       max(order_purchase_timestamp) as last_order  
from `orders.Orders`.
```



C: Count the number of Cities and States in our dataset.

Answer:

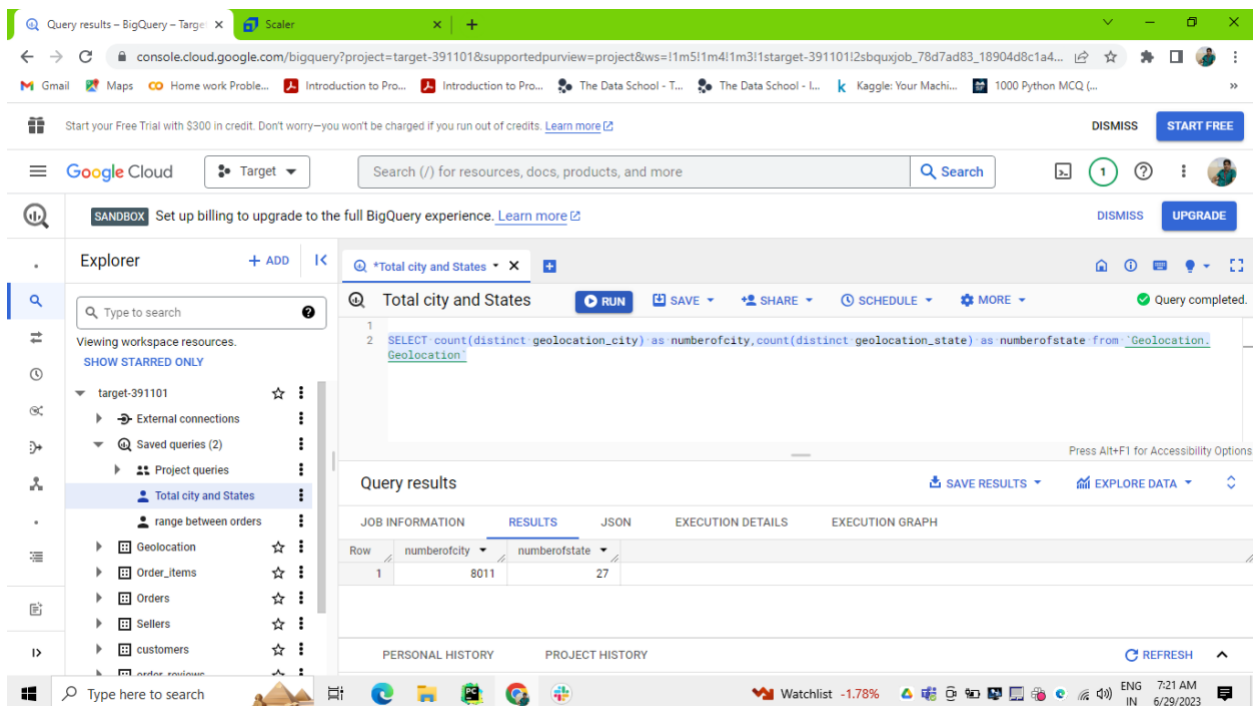
As the question clearly mentions to count the number of Cities and States.

There are 8011 Cities and 27 States in the given dataset.

We can perform a count distinct on geolocation_city and geolocation_state columns from the geolocation table to extract the number of cities and state.

The total number of cities = 8011

The total number of states = 27



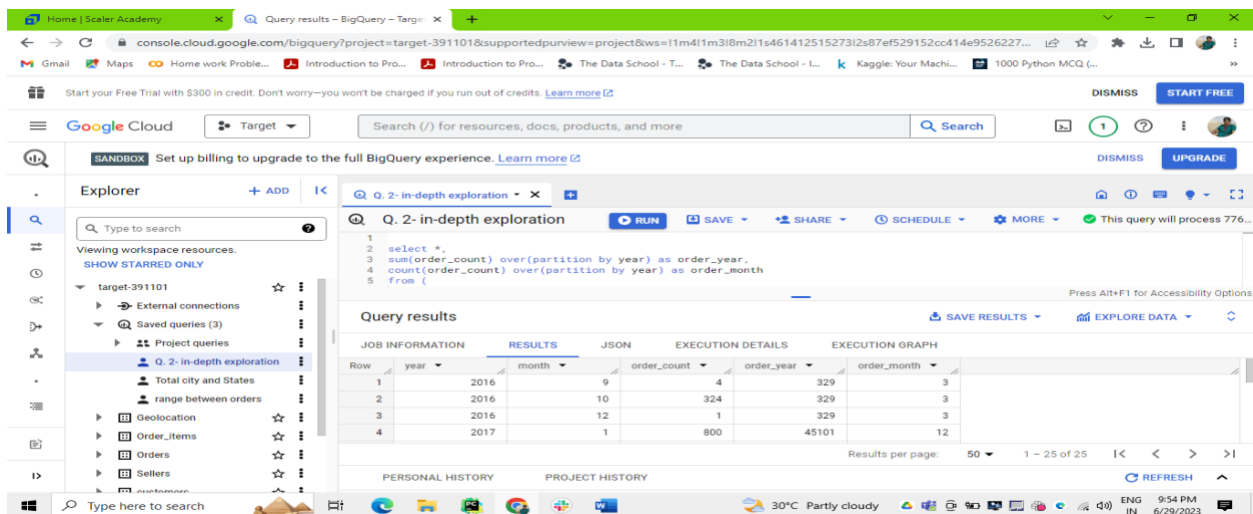
Insights: Executing this query helps you understand the breadth of city and state coverage in the database, providing insights into the diversity and geographic distribution of the recorded locations.

Question-2: In-depth Exploration:

A: Is there a growing trend in the no. of orders placed over the past years?

Answer:

```
select *,
sum(order_count) over(partition by year) as order_year,
count(order_count) over(partition by year) as order_month
from (
  SELECT
    EXTRACT(YEAR FROM order_purchase_timestamp) AS year,
    EXTRACT(MONTH FROM order_purchase_timestamp) AS month,
    COUNT(*) AS order_count,
  FROM `Orders.Orders`
  GROUP BY 1,2
  ORDER BY 1,2)
order by year asc, month asc;
```



The screenshot shows the Google Cloud BigQuery console interface. The query editor on the right contains the SQL query from the previous block. Below the editor, the 'Query results' section displays a table with 5 columns: Row, year, month, order_count, and order_year. The table shows data for the years 2016 and 2017, with the cumulative sum of order counts (order_year) increasing over time. The 'order_month' column is also present but appears to be a duplicate of 'order_count' in the data shown.

Row	year	month	order_count	order_year	order_month
1	2016	9	4	329	3
2	2016	10	324	329	3
3	2016	12	1	329	3
4	2017	1	800	45101	12

Insights: We can obtain a result set that includes the original year, month, and order count columns, along with the cumulative sum of order counts for each year (order_year) and the count of order counts for each year and month (order_month). This provides insights into the distribution and trends of order counts over time.

B: Can we see some kind of monthly seasonality in terms of the no. of orders being placed?

Answer:

```
select *,
sum(order_count) over(partition by year) as order_year,
count(order_count) over(partition by year) as order_month
from (
  SELECT
    EXTRACT(YEAR FROM order_purchase_timestamp) AS year,
    EXTRACT(MONTH FROM order_purchase_timestamp) AS month,
    COUNT(*) AS order_count,
```

```
FROM `Orders.Orders`
GROUP BY 1,2
ORDER BY 1,2)
order by year asc, month asc;
```

The screenshot shows the Google Cloud BigQuery console interface. The query editor on the left contains the following SQL:

```
select *,
sum(order_count) over(partition by year) as order_year,
count(order_count) over(partition by year) as order_month
```

The query results are displayed in a table with the following data:

Row	year	month	order_count	order_year	order_month
1	2016	9	4	329	3
2	2016	10	324	329	3
3	2016	12	1	329	3
4	2017	1	800	45101	12
5	2017	2	1780	45101	12
6	2017	3	2682	45101	12

Insights: Here we couldn't see any seasonality because there was no relation between the given years. Also, one more reason we couldn't embark on seasonality can be limited database.

C: During what time of the day, do the Brazilian customers mostly place their orders? (Dawn, Morning, Afternoon or Night)

0-6 hrs : Dawn

7-12 hrs : Mornings

13-18 hrs : Afternoon

19-23 hrs : Night

Answer:

```
SELECT
CASE
    WHEN EXTRACT(HOUR FROM order_purchase_timestamp) BETWEEN 0 AND 6 THEN 'Dawn'
    WHEN EXTRACT(HOUR FROM order_purchase_timestamp) BETWEEN 7 AND 12 THEN 'Morning'
    WHEN EXTRACT(HOUR FROM order_purchase_timestamp) BETWEEN 13 AND 18 THEN
'Afternoon'
    WHEN EXTRACT(HOUR FROM order_purchase_timestamp) BETWEEN 19 AND 23 THEN 'Night'
END AS ordeTime,
COUNT(*) AS order_count
FROM `Orders.Orders`
```

```
group by 1
ORDER BY
order_count DESC;
```

The screenshot shows the Google Cloud BigQuery console interface. The query editor displays a SQL query that categorizes orders by time of day. The query results are shown in a table with columns 'orderTime' and 'order_count'.

Query:

```
SELECT
CASE
WHEN EXTRACT(HOUR FROM order_purchase_timestamp) BETWEEN 0 AND 6 THEN 'Dawn'
WHEN EXTRACT(HOUR FROM order_purchase_timestamp) BETWEEN 7 AND 12 THEN 'Morning'
WHEN EXTRACT(HOUR FROM order_purchase_timestamp) BETWEEN 13 AND 18 THEN 'Afternoon'
WHEN EXTRACT(HOUR FROM order_purchase_timestamp) BETWEEN 19 AND 23 THEN 'Night'
END AS orderTime,
COUNT(*) AS order_count
FROM `target-391101.orders`
GROUP BY 1
ORDER BY 2 DESC;
```

Query results:

Row	orderTime	order_count
1	Afternoon	38135
2	Night	28331
3	Morning	27733
4	Dawn	5242

Insights: This query can help identify patterns and trends in customer ordering behavior based on the time of day.

Question – 3 : Evolution of E-commerce orders in the Brazil region:

A: Get the month on month no. of orders placed in each state

Answer:

```
select c.customer_state,extract(year from o.order_purchase_timestamp) as year,
extract(month from o.order_purchase_timestamp) as month,
COUNT(*) as count_order_per_state
from `Orders.Orders` o
join `customers.customers` c on o.customer_id = c.customer_id
group by 1,2,3
order by 2,3
```

The screenshot shows the Google Cloud BigQuery console. The query editor contains the following SQL:

```
1 select o.customer_state, extract(year from o.order_purchase_timestamp) as year,
2 extract(month from o.order_purchase_timestamp) as month,
```

The query results are displayed in a table with the following data:

Row	customer_state	year	month	count_order_per_state
1	RR	2016	9	1
2	RS	2016	9	1
3	SP	2016	9	2
4	SP	2016	10	113
5	RS	2016	10	24
6	RJ	2016	10	56

Insights: The result includes the customer state, year, month, and the corresponding count of orders. This information can be useful in analyzing order patterns, identifying peak months, and understanding customer behavior based on location.

B: How are the customers distributed across all the states?

Answer:

```
SELECT customer_state, COUNT(customer_unique_id) AS no_of_customers
from `customers.customers`
group by 1
```

The screenshot shows the Google Cloud BigQuery console. The query editor contains the following SQL:

```
1 SELECT customer_state, COUNT(customer_unique_id) AS no_of_customers
2 from `customers.customers`
3 group by 1
```

The query results are displayed in a table with the following data:

Row	customer_state	no_of_customers
1	RN	485
2	CE	1336
3	RS	5466
4	SC	3637
5	SP	41746

Insights: We can gain insights into the number of unique customers in each state. The result will include the customer state and the corresponding count of customers. This information can be useful in understanding customer distribution across different states and identifying regions with a higher or lower concentration of customers.

Question.4: Impact on Economy: Analyze the money movement by e-commerce by looking at order prices, freight and others.

A: Get the % increase in the cost of orders from year 2017 to 2018 (include months between Jan to Aug only).

You can use the "payment_value" column in the payments table to get the cost of orders.

Answer:

```
select
    (sum(case when extract(Year from b.order_purchase_timestamp) = 2018 then
a.payment_value end)-
    sum(case when extract(Year from b.order_purchase_timestamp) = 2017 then
a.payment_value end))/
    sum(case when extract(Year from b.order_purchase_timestamp) = 2017 then
a.payment_value end)*100 as perc_inc
from `payments.payments` a
join `Orders.Orders` b on a.order_id=b.order_id
where extract(Month from b.order_purchase_timestamp) between 1 and 8;
```

The screenshot shows the Google Cloud BigQuery console interface. On the left, the 'Explorer' pane displays a list of workspace resources, including 'Saved queries (6)' and 'Project queries'. The main area shows a SQL query titled 'Untitled 4' with the following code:

```
1 select
2     (sum(case when extract(Year from b.order_purchase_timestamp) = 2018 then a.payment_value end)-
3     sum(case when extract(Year from b.order_purchase_timestamp) = 2017 then a.payment_value end))/
4     sum(case when extract(Year from b.order_purchase_timestamp) = 2017 then a.payment_value end)*100 as perc_inc
5 from `payments.payments` a
```

Below the query editor, the 'Query results' section is visible, showing a table with one row and one column, 'perc_inc', with the value 136.9768716466... The interface also includes a top navigation bar with 'Google Cloud' and 'Target' tabs, and a bottom status bar showing the system clock and weather.

Insights: We can obtain insights into the percentage increase in payment value between 2017 and 2018 for orders placed in the specified time period. The result will include the percentage increase as the column perc_inc.

B: Calculate the Total & Average value of order price for each state.

Answer:

```
SELECT DISTINCT c.customer_state, SUM(p.payment_value)OVER(PARTITION BY
c.customer_state) AS total,
AVG(p.payment_value)OVER(PARTITION BY c.customer_state) AS avg
FROM `customers.customers` c
JOIN `Orders.Orders` o ON c.customer_id = o.customer_id
```

```
JOIN `payments.payments` p ON o.order_id = p.order_id
ORDER BY c.customer_state
```

The screenshot shows the Google Cloud BigQuery console interface. On the left is the Explorer pane with a search bar and a list of project queries. The main area displays a query titled 'Untitled 3' with the following SQL:

```
1 SELECT DISTINCT c.customer_state, SUM(p.payment_value)OVER(PARTITION BY c.customer_state) AS total,
2 AVG(p.payment_value)OVER(PARTITION BY c.customer_state) AS avg
```

The query results are shown in a table with the following data:

Row	customer_state	total	avg
1	AC	19680.62	234.2930952380...
2	AL	96962.06	227.0774238875...
3	AM	27966.93	181.6034415584...
4	AP	16262.8	232.3257142857...
5	BA	616645.82	170.8160166204...
6	CE	279464.03	199.9027396280...

The interface also shows a 'Query completed' status and a 'Results per page' dropdown set to 50.

Insights: The result includes the customer state, the total payment value for that state, and the average payment value for that state. This information can help analyze payment patterns and spending behavior across different states, providing insights into customer preferences and regional variations.

C: Calculate the Total & Average value of order freight for each state.

Answer:

```
SELECT DISTINCT c.customer_state, SUM(oi.freight_value)OVER(PARTITION BY
c.customer_state) AS total,
AVG(oi.freight_value)OVER(PARTITION BY c.customer_state) AS avg
FROM `customers.customers` c
JOIN `Orders.Orders` o ON c.customer_id = o.customer_id
JOIN `Order_items.Order_items` oi ON o.order_id = oi.order_id
ORDER BY c.customer_state
```


The screenshot shows the Google Cloud BigQuery console. The query editor displays the following SQL query:

```
1 SELECT DISTINCT c.customer_state, SUM(o1.freight_value) OVER (PARTITION BY c.customer_state) AS total,
2 AVG(o1.freight_value) OVER (PARTITION BY c.customer_state) AS avg
3 FROM `customers.customers` c
```

The query results are displayed in a table with the following data:

Row	customer_state	total	avg
1	AC	3686.75	40.07336956521...
2	AL	15914.59	35.84367117117...
3	AM	5478.89	33.20539393939...
4	AP	2788.5	34.00609756097...
5	BA	100156.68	26.36395893656...

Insights: The result includes the customer state, total & average value of order freight for each state. This information can help analyze payment patterns and spending behavior across different states, providing insights into customer preferences and regional variations.

Question-5: Analysis based on sales, freight and delivery time.

A: Find the no. of days taken to deliver each order from the order's purchase date as delivery time.

Also, calculate the difference (in days) between the estimated & actual delivery date of an order.

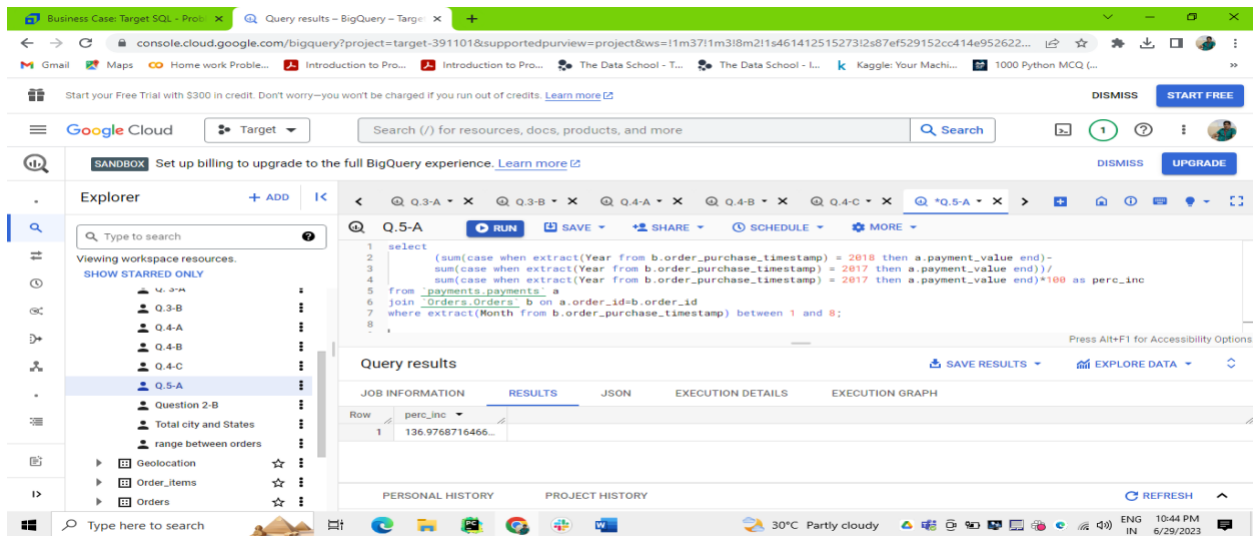
Do this in a single query.

You can calculate the delivery time and the difference between the estimated & actual delivery date using the given formula:

- time_to_deliver = order_delivered_customer_date - order_purchase_timestamp
- diff_estimated_delivery = order_estimated_delivery_date - order_delivered_customer_date

Answer:

```
select
    (sum(case when extract(Year from b.order_purchase_timestamp) = 2018 then
a.payment_value end)-
    sum(case when extract(Year from b.order_purchase_timestamp) = 2017 then
a.payment_value end))/
    sum(case when extract(Year from b.order_purchase_timestamp) = 2017 then
a.payment_value end)*100 as perc_inc
from `payments.payments` a
join `Orders.Orders` b on a.order_id=b.order_id
where extract(Month from b.order_purchase_timestamp) between 1 and 8;
```



Insights: By executing this query, we can obtain insights into the percentage increase in payment value between 2017 and 2018 for orders placed in the specified time period. The result will include the percentage increase as the column `perc_inc`. This can help analyze the growth or decline in payment amounts over time and identify trends in customer spending behavior.

B: Find out the top 5 states with the highest & lowest average freight value.

Answer:

Top 5 states with the highest average freight value.

```
SELECT *, dense_rank()over( ORDER BY avg_freight_value DESC) AS state_rank,
from
(
SELECT c.customer_state,AVG(oi.freight_value) AS avg_freight_value
FROM `Order_items.Order_items` oi
JOIN `Orders.Orders` o ON o.order_id = oi.order_id
JOIN `customers.customers` c on c.customer_id = o.customer_id
GROUP BY 1
ORDER BY 1
)
order by state_rank
LIMIT 5
```

The screenshot shows the Google Cloud BigQuery console. The query editor displays the following SQL:

```
SELECT *, dense_rank()over( ORDER BY avg_freight_value) AS state_rank,
from
(
SELECT c.customer_state,AVG(oi.freight_value) AS avg_freight_value
FROM `Order_items.Order_items` oi
JOIN `Orders.Orders` o ON o.order_id = oi.order_id
JOIN `customers.customers` c on c.customer_id = o.customer_id
GROUP BY 1
ORDER BY 1
)
order by state_rank
LIMIT 5
```

The query results table shows the following data:

Row	customer_state	avg_freight_value	state_rank
1	RR	42.98442307692...	1
2	PB	42.72380398671...	2
3	RO	41.06971223021...	3
4	AC	40.07336956521...	4
5	PI	39.14797047970...	5

Bottom 5 states with highest freight value.

```
SELECT *, dense_rank()over( ORDER BY avg_freight_value) AS state_rank,
from
(
SELECT c.customer_state,AVG(oi.freight_value) AS avg_freight_value
FROM `Order_items.Order_items` oi
JOIN `Orders.Orders` o ON o.order_id = oi.order_id
JOIN `customers.customers` c on c.customer_id = o.customer_id
GROUP BY 1
ORDER BY 1
)
order by state_rank
LIMIT 5
```

The screenshot shows the Google Cloud BigQuery console. The query editor displays the following SQL:

```
ORDER BY 1
)
order by state_rank
```

The query results table shows the following data:

Row	customer_state	avg_freight_value	state_rank
1	SP	15.14727539041...	1
2	PR	20.53165156794...	2
3	MG	20.63016680630...	3
4	RJ	20.96092393168...	4
5	DF	21.04135494596...	5

C: Find out the top 5 states with the highest & lowest average delivery time.

Answer:

Top 5 states with the highest average delivery time.

```

SELECT *, dense_rank()over( ORDER BY time_to_deliver DESC) AS ranked,
from
(
SELECT c.customer_state, AVG(EXTRACT(DAY FROM o.order_delivered_carrier_date)-
EXTRACT(DAY FROM o.order_purchase_timestamp) ) AS time_to_deliver
FROM `Orders.Orders` o
JOIN `customers.customers` c on c.customer_id = o.customer_id
GROUP BY 1
ORDER BY 1
)
order by ranked
LIMIT 5

```

The screenshot shows the Google Cloud BigQuery console. The query results are displayed in a table with columns: Row, customer_state, time_to_deliver, and ranked. The results show the bottom 5 states with the lowest average delivery time.

Row	customer_state	time_to_deliver	ranked
1	AP	2.134328358208...	1
2	AC	0.814814814814...	2
3	CE	0.796373292867...	3
4	PI	0.652892561983...	4
5	MS	0.646306818181...	5

Bottom 5 states with the lowest average delivery time.

```

SELECT *, dense_rank()over( ORDER BY time_to_deliver) AS ranked,
from
(
SELECT c.customer_state, AVG(EXTRACT(DAY FROM o.order_delivered_carrier_date)-
EXTRACT(DAY FROM o.order_purchase_timestamp) ) AS time_to_deliver
FROM `Orders.Orders` o
JOIN `customers.customers` c on c.customer_id = o.customer_id
GROUP BY 1
ORDER BY 1
)
order by ranked
LIMIT 5

```

The screenshot shows the Google Cloud BigQuery console interface. On the left is the Explorer pane showing a list of workspace resources. The main area displays a query result for 'Untitled 3'. The query is 'order by ranked' and the results are shown in a table with 5 rows. The columns are 'customer_state', 'time_to_deliver', and 'ranked'. The states are ranked from fastest to slowest delivery.

Row	customer_state	time_to_deliver	ranked
1	TO	-0.99280575539...	1
2	MA	-0.21224489795...	2
3	RR	-0.13333333333...	3
4	RN	0.012474012474...	4
5	AM	0.027210884353...	5

D: Find out the top 5 states where the order delivery is really fast as compared to the estimated date of delivery.

You can use the difference between the averages of actual & estimated delivery date to figure out how fast the delivery was for each state.

Answer:

```
SELECT *, DENSE_RANK()OVER(ORDER BY a DESC) as b
FROM
(
SELECT c.customer_state,AVG(EXTRACT(DAY FROM o.order_delivered_customer_date)),
AVG(EXTRACT(DAY FROM o.order_estimated_delivery_date)),
(AVG(EXTRACT(DAY FROM o.order_estimated_delivery_date))-AVG(EXTRACT(DAY FROM
o.order_delivered_customer_date))) AS a
FROM `Orders.Orders` o
JOIN `customers.customers` c ON o.customer_id = c.customer_id
WHERE o.order_status = 'delivered'
GROUP BY 1
ORDER BY 1)
ORDER BY b
LIMIT 5
```

The screenshot shows the Google Cloud BigQuery console. The Explorer on the left lists workspace resources including 'Q.4-B', 'Q.4-C', 'Q.5-A', 'Q.5-B', 'Q.5-C', 'Question 2-B', 'Total city and States', and 'range between orders'. The main panel displays a query titled 'Untitled 3' with the following SQL:

```
1 SELECT *, DENSE_RANK() OVER (ORDER BY a DESC) as b
2 FROM
```

The query results are shown in a table with 5 rows and 6 columns: 'customer_state', 'f0_', 'f1_', 'a', 'b', and an unnamed column. The data is as follows:

Row	customer_state	f0_	f1_	a	b
1	AM	14.08275862068...	16.06206896551...	1.979310344827...	1
2	AC	15.5499999999999...	16.125000000000...	0.575000000000...	2
3	AP	17.13432835820...	17.56716417910...	0.432835820895...	3
4	PA	15.79598308668...	16.12156448202...	0.325581395348...	4
5	RS	15.79491017964...	15.73077642656...	-0.06413375307...	5

Question- 6: Analysis based on the payments:

A: Find the month on month no. of orders placed using different payment types.

Answer:

```
SELECT
EXTRACT(YEAR FROM o.order_purchase_timestamp) AS year,
EXTRACT(MONTH FROM o.order_purchase_timestamp) AS month, p.payment_type,
case
when p.payment_type = 'UPI' then count(p.payment_type)
when p.payment_type = 'debit_card' then count(p.payment_type)
when p.payment_type = 'credit_card' then count(p.payment_type)
when p.payment_type = 'voucher' then count(p.payment_type)
when p.payment_type = 'not_defined' then count(p.payment_type)
end as count_pay
FROM `Orders.Orders` o
JOIN `payments.payments` p ON o.order_id = p.order_id
GROUP BY 1, 2, 3
ORDER BY 1,2
LIMIT 1000
```

The screenshot shows the Google Cloud BigQuery console with the query results for the payment analysis query. The query is titled 'Untitled 3' and the results are displayed in a table with 14 rows and 5 columns: 'year', 'month', 'payment_type', and 'count_pay'. The data is as follows:

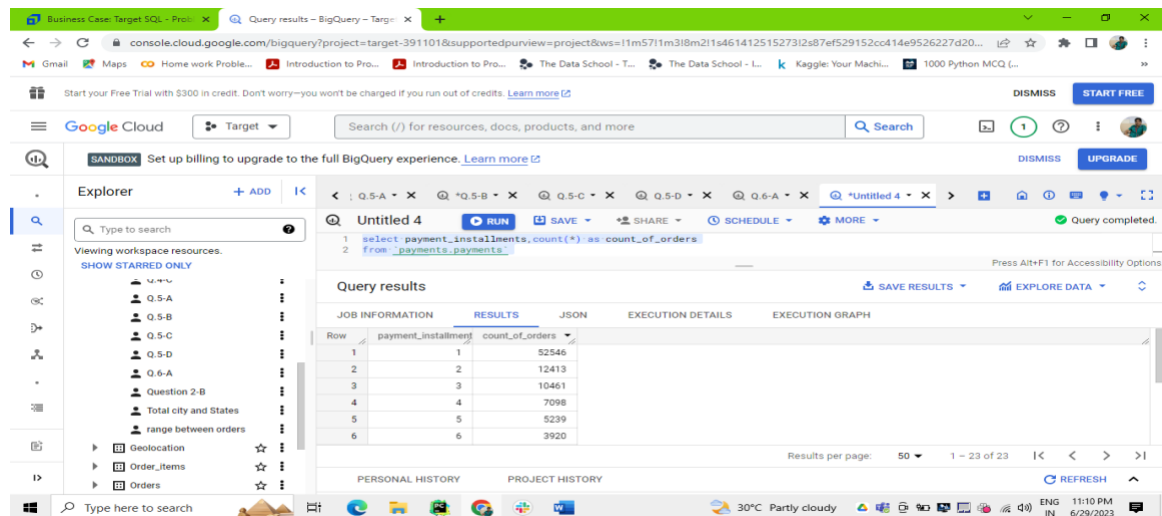
Row	year	month	payment_type	count_pay
9	2017	1	credit_card	583
10	2017	1	debit_card	9
11	2017	2	credit_card	135A
12	2017	2	debit_card	13
13	2017	2	UPI	398
14	2017	2	voucher	119

Insights: The result includes the year, month, payment type, and count of occurrences for that payment type. This information can help analyze the distribution of payment types over time and identify any patterns or trends in customer payment preferences.

B: Find the no. of orders placed on the basis of the payment installments that have been paid.

Answer:

```
select payment_installments, count(*) as count_of_orders
from `payments.payments`
GROUP BY 1
HAVING payment_installments >=1
```



The screenshot shows the Google Cloud BigQuery console interface. The query editor on the left contains the SQL query: `select payment_installments, count(*) as count_of_orders from `payments.payments` GROUP BY 1 HAVING payment_installments >=1`. The query results are displayed in a table with 6 rows. The table has two columns: `payment_installment` and `count_of_orders`. The results are as follows:

Row	payment_installment	count_of_orders
1	1	52546
2	2	12413
3	3	10461
4	4	7098
5	5	5239
6	6	3920

Insights: The result includes the payment_installments value and the count of orders associated with that installment value. This information can help analyze the usage of different payment installment options by customers and understand the popularity of certain installment plans.

Summary:

The provided tasks involve exploring and analyzing an e-commerce dataset from Brazil. The analysis encompasses various aspects such as customer information, order trends, economic impact, delivery performance, and payment patterns. By examining data types, time range, city/state counts, order trends, seasonal patterns, preferred ordering times, state-wise order analysis, economic impact analysis, delivery time analysis, and payment analysis, valuable insights can be gained to inform business decisions and strategies.

Suggestions:

To drive growth and improve customer satisfaction, I would suggest the company focus on enhancing customer segmentation for targeted marketing, optimizing delivery processes to ensure timely and efficient shipments, offering flexible payment options to cater to diverse preferences, monitoring and addressing freight costs, capitalizing on seasonal trends through strategic planning, utilizing data-driven insights for predictive analysis, enhancing customer support systems, fostering customer loyalty through incentives, staying updated with industry trends, and conducting regular

performance evaluations to identify areas for improvement and align business strategies with customer needs.